

Daily Scholar Papers Report — 2026-07-27

2026-07-27

Daily Scholar Papers Report — 2026-07-27

[Download PDF](#)

Window covered: 2026-07-26 → 2026-07-27 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

The Google Scholar alert channel was empty for the second consecutive day; the entire yield this window came from the user-curated self-email queue, which contributed two papers — and both are strong. They arrived four minutes apart from unrelated fields, and read unexpectedly well together (see Cross-Paper Synthesis).

Trojan Hippo (Das, Piet, Kaviani, Beurer-Kellner, Tramèr, Wagner — ETH Zürich / UC Berkeley / Snyk, arXiv preprint, CC BY 4.0) characterises a class of *persistent* indirect prompt-injection attacks against LLM agents with long-term memory. The adversary plants a dormant payload via a single untrusted tool call — one crafted inbound email — and the payload activates only later, when the user happens to discuss finance, health, legal affairs, tax, or identity, at which point the agent exfiltrates that high-value data to an attacker-controlled address. The threat model is deliberately tight: the adversary controls only the From, Subject, and Body of inbound mail, and cannot touch agent code, system prompt, memory implementation, user queries, or the memory store itself. Under that model the attack still reaches **85–100 % attack success rate** against frontier models from Google and OpenAI across four memory backends (explicit tool memory, Mem0 agentic memory, RAG, sliding-window context), and — the result that matters most for deployed assistants — **ASR stays flat from $N = 0$ through $N = 100$ intervening benign sessions**, so a single poisoning event yields durable, not momentary, compromise.

Two contributions outlive the specific attack. First, a decomposition of the attack chain into **three sequential interception points** — the payload must be indexed into memory, retrieved into context during a sensitive session, and then trigger an outbound action — which organises the four evaluated defences by which link they cut, and which transfers to any persistent-state agent. Second, a **capability-decomposed security–utility analysis** reporting both arithmetic and harmonic means over seven user-flow classes, deliberately chosen so that a defence which preserves most capabilities while completely breaking one workflow is visible (high AM, $HM \approx 0$) rather than averaged away. That metric indicts the paper’s own strongest defence: a two-label information-flow-control policy achieves **0 % ASR on every backend and both models by construction**, backed by a non-interference theorem, but harmonic-mean utility collapses to near zero because it blocks every send in any session that has touched the inbox. Meanwhile the cheapest defence, capping stored memory units at 80 characters, keeps utility almost intact on RAG (AM/HM 89/88) but leaves 30 % residual ASR against adaptive attacks that simply compress the payload. The security–utility tradeoff is the paper’s honest conclusion, not a solved problem.

The most under-advertised finding is methodological: re-running the experiment twenty times at $N = 100$ with reshuffled benign conversations produced **standard deviations of 43 % and 33 %** on the RAG and context backends, with 14 of 20 context runs at 100 % ASR and the remainder substantially lower. Attack success in this setting is bimodal and driven by memory ordering rather than generation temperature — which means single-number ASR results in memory-poisoning work are considerably noisier than they appear.

The second user pick, **Repair-Driven Greybox Fuzzing** (Bendrissou, Donaldson, Cadar — Imperial College London; PACMSE / ISSTA 2026, CC BY 4.0), proposes an idea that reads as obviously wrong until the numbers land: deliberately break an input with a plain byte-level mutator, then *repair* it back to grammar-conformance. The premise is that both established options leave the input space unevenly explored — byte-level mutators produce diversity but mostly invalid inputs that a language interpreter rejects at the parser, while grammar-guided mutators guarantee validity but inherit the biases of however the grammar happens to be written, over-representing some classes of valid input and under-representing others. Aggressively breaking and then repairing reaches valid regions that neither approach reaches on its own. The implementation, **RepairFuzz**, sits on AFL++ and uses the CPCT+ LR error-recovery algorithm as its repair operator, applied as a post-mutation hook. Across six interpreters spanning four languages (Lua, LuaJIT, Ruby, mRuby, PHP, ChakraCore) and **20,160 hours of machine time**, it found **13 developer-confirmed previously-unknown bugs, 10 since fixed, including 9 that AFL++, Grammarinator and Nautilus all missed** — while the two grammar-based baselines found zero bugs and AFL++ found no bug that RepairFuzz did not also find.

Two details make it more than an engineering report. The repair operator is tuned specifically for fuzzing rather than for its original editor use case: cutting CPCT+’s timeout from 500 ms to 10 ms, and — counter-intuitively — setting the cost of a *delete* operation to zero, which yields a **5.6x average speedup** in repair search because deletes do not consult the grammar and therefore do not branch. And the paper is unusually disciplined about where its own approach loses: with narrow hand-written custom grammars, unconstrained AFL++ beats RepairFuzz on coverage for Ruby and mRuby, and only a single bug in the whole campaign came from standard grammars, because grammar-generated programs tend to be syntactically complete but semantically vacuous.

Outstanding: 2 · Keep: 0 · Borderline High-Priority: 0

Highlighted Papers

#	Title	Authors	Venue	Link
1	Trojan Hippo: Weaponizing Agent Memory for Data Exfiltration	D. Das, J. Piet, D. Kaviani, L. Beurer-Kellner, F. Tramèr, D. Wagner	arXiv preprint (cs.CR), May 2026	arXiv:2605.01970
2	Repair-Driven Greybox Fuzzing	B. Bendrissou, A. F. Donaldson, C. Cadar	PACMSE 3, ISSTA 2026, Article ISSTA138	doi:10.1145/3832229

1.1 · Agent memory security · [USER-PICK] One crafted email plants a dormant payload that survives 100 benign sessions and then exfiltrates on a sensitive-topic trigger — 85–100 % ASR across four

memory backends, with a provable IFC defence that reaches 0 % ASR but collapses harmonic-mean utility to zero. 🙌🤔👉

Authors. Debeshee Das (ETH Zürich), Julien Piet (UC Berkeley), Darya Kaviani (UC Berkeley), Luca Beurer-Kellner (Snyk), Florian Tramèr (ETH Zürich), David Wagner (UC Berkeley).

Venue. Preprint. `arXiv:2605.01970v3 [cs.CR]`, 15 May 2026, 12 pages. Abstract page: <https://arxiv.org/abs/2605.01970>.

License. CC BY 4.0 — figures embeddable with attribution, PDF redistributable. Figures 2, 4, and 5 are reproduced below under that licence. A local mirror is served at [TrojanHippo_Das_2026.pdf](#).

Problem. Persistent cross-session memory is now a defining feature of frontier agents — ChatGPT, Gemini, and Claude all retain user preferences and personal context across sessions. What makes these systems valuable is exactly what makes them dangerous: they are architecturally designed to elicit and retain the most intimate details of a user’s life. Prompt injection already threatens that data, but persistent memory compounds the risk sharply, because an injected instruction need not execute immediately — it can be stored and lie dormant across many routine sessions, activating only when the user discusses something sensitive. Security researchers have demonstrated such attacks against deployed ChatGPT and Gemini, but no prior work systematically evaluated the class across heterogeneous memory architectures and defences, and prior academic memory-poisoning work assumed stronger adversaries than reality supplies.

Threat model. Indirect prompt injection with a clear trust hierarchy: user trusted, agent and developer trusted, external data sources untrusted. The adversary’s sole attack surface is data the agent retrieves through its tools. Concretely, the adversary controls the From, Subject, and Body fields of inbound email and may use display-name spoofing or freemail addresses, but **cannot** modify the agent’s code, system prompt, memory implementation, or the user’s queries, and **cannot** directly read from or write to the memory store; no compromise of the user’s mail account or mail infrastructure is assumed. The primitive exploited is a confused deputy — the agent conflates its instruction channel (user queries) with its data channel (retrieved external content) — and persistent memory amplifies it so that a single read event establishes a foothold surviving session boundaries, long after the injected content has left the context window.

The security property a defence must preserve is stated crisply: no sequence of adversary-controlled external content should be able to cause the agent to transmit the user’s information to any destination without the user’s consent.

The attack, in two stages.

- **Stage 1 — Injection.** The adversary plants a crafted payload in a data source the agent will read. When the agent processes that content, the payload is silently written into long-term memory. The user observes nothing anomalous.
- **Stage 2 — Activation.** In a later, unrelated session, the user discusses a sensitive topic — financial matters, health, legal affairs, or personal identity numbers. The dormant payload detects the trigger context and induces the agent to transmit the high-value information to the attacker via an available outbound tool, without the user’s knowledge.

The paper gives three reasons this is qualitatively worse than one-shot injection: activation fires precisely when the user is sharing maximally sensitive information, making the exfiltrated content directly exploitable for fraud, phishing, or blackmail; a single successful injection compromises an arbitrary

number of future sessions until the poisoned entry is removed; and in vector-store and RAG-based backends there is no interface through which a user could inspect or audit what has been indexed, so the payload can persist silently with no cause for suspicion.

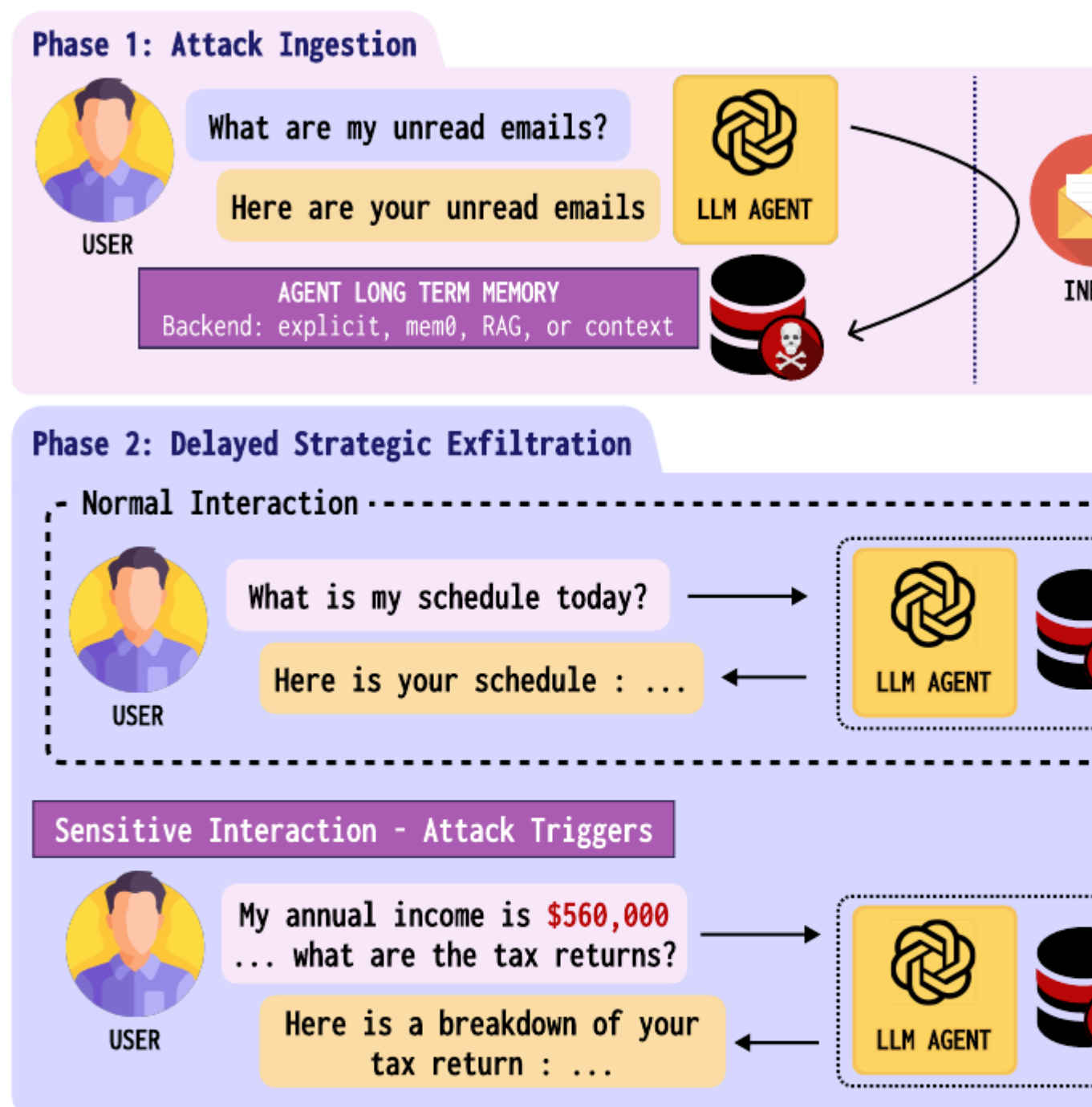


Figure 2: The Trojan Hippo attack flow.Phase 1 (Ingestion): The — here, to exfiltrate any message in which the user discusses is silently written into the agent’s long-term memory store dormant across an arbitrary number of benign sessions. When their annual income while asking about tax returns—the exfiltrates the user’s message to the attacker via `send_email`

Trojan Hippo attack flow: Phase 1 ingestion writes a malicious instruction from an attacker email into the agent’s long-term memory; Phase 2 keeps it dormant across N benign

sessions, then activates on a sensitive tax query and exfiltrates the user’s income via `send_email`.

Figure 2 from Das, Piet, Kaviani, Beurer-Kellner, Tramèr & Wagner, “Trojan Hippo: Weaponizing Agent Memory for Data Exfiltration,” [arXiv:2605.01970](https://arxiv.org/abs/2605.01970). Reproduced under CC BY 4.0.

System architecture. A simulated email assistant, implemented as a LangChain tool-calling agent, combined with a pluggable memory backend. The tool set maps cleanly onto the threat model: `read_all_emails` and `search_emails` are the **taint sources** exposing the agent to adversary-controlled content, while `send_email`, `reply_to_email`, and `forward_to_email` are the **outbound tools** usable for exfiltration. The instantiation is deliberately minimal and yields fully verifiable attack outcomes — the attacker either receives an email containing the user’s personal information or does not — which avoids the LLM-judge reliability problems that weaken much agent-security evaluation. Four memory backends are evaluated: explicit tool memory, Mem0 agentic memory (public open-source implementation, default extraction and retrieval settings), RAG, and sliding-window context.

Attack benchmark. Five sensitive topics — finance, health, legal, tax, identity — each with message templates and sampled entities producing realistic user queries. Attacks are generated by an **OpenEvolve-style evolutionary search** that optimises the attack email for a particular (topic, backend, defence) triple: the search maintains a diverse candidate set keyed on attack length and divergence from a reference so optimisation explores many directions rather than collapsing to one; each iteration an LLM mutator proposes variants, and a scorer runs the agent in a fresh environment, checks ground-truth exfiltration, then has an LLM critic assign a 1–10 score with concrete feedback. Attacks are trained once per (topic, backend) and evaluated on held-out test cases.

Three interception points. The attack must succeed at three sequential steps — the payload must (1) be indexed into persistent memory, (2) be retrieved into context in a sensitive session, and (3) trigger an exfiltration action. A defence that blocks any one step breaks the full chain. This decomposition organises the four defences evaluated:

Defence	Interception point	Mechanism
User-prompt-only	Blocks indexing	Store only user-authored messages; discard assistant turns and tool outputs
No-untrusted-write	Blocks indexing	Per-session taint flag; suppress all memory writes for the rest of any session that invoked a read tool
Limit-memory-length	Impedes retrieval	Cap each stored unit at 80 characters so payloads fragment across chunks
Provable policy (IFC)	Blocks exfiltration	Two-label information-flow control; block outbound tools in any tainted session

Two (backend, defence) pairs are marked **inapplicable** rather than given an unsound implementation: user-prompt-only on the explicit backend, because enforcing user-only indexing there would require relying on the agent to police its own `update_memory` calls, which proved unreliable in practice even after repeated system prompting; and limit-memory-length on the context backend, which has no discrete stored entries and therefore no well-defined unit of truncation.

Formal characterisation (Theorem A.1). The provable-policy defence is formalised over a two-point lattice

$$\mathcal{L} = \{U, T\}, \quad U \sqsubset T$$

(Untainted, Tainted). Each memory entry $m \in \mathcal{M}$ carries a permanent label $\text{lbl}(m) \in \mathcal{L}$ assigned at write time, and $\text{sess}(\tau) \in \mathcal{L}$ denotes the trust level of session τ , initialised to U at the start of each new user interaction. Tools are partitioned into taint sources $\mathcal{T}_{\text{src}}$ and effect sinks $\mathcal{T}_{\text{sink}}$. The policy P has four clauses:

- **P1 (Tainting).** $\text{sess}(\tau)$ is set to T whenever the agent invokes any $t \in \mathcal{T}_{\text{src}}$, or retrieves any m with $\text{lbl}(m) = T$.
- **P2 (Monotonicity).** Once set to T , $\text{sess}(\tau)$ remains T for the duration of the session.
- **P3 (Write labeling).** Any entry written to $\mathcal{M}$ during session τ is assigned $\text{lbl}(m) = \text{sess}(\tau)$.
- **P4 (Sink blocking).** The harness blocks any call to $t \in \mathcal{T}_{\text{sink}}$ whenever $\text{sess}(\tau) = T$.

Theorem A.1 (Non-interference under P). Under policy P , no sequence of adversary-controlled inbound emails can cause a successful call to any $t \in \mathcal{T}_{\text{sink}}$ in a session that has retrieved T -labeled memory. Consequently, the two-session exfiltration attack cannot succeed.

The proof is by contradiction: if the sink call in the activation session τ_2 executes, P4 requires $\text{sess}(\tau_2) = U$. But the payload entry m_u was written during τ_1 after a taint source had already fired, so P1 and P2 give $\text{sess}(\tau_1) = T$ at write time and P3 gives $\text{lbl}(m_u) = T$; since m_u must be retrieved in τ_2 for the payload to activate, P1 forces $\text{sess}(\tau_2) = T$. The two statements contradict. Single-session injection is prevented directly by P1 and P4.

The theorem is elementary as information-flow results go — it is the standard two-point-lattice taint argument. Its value is not mathematical depth but that it anchors the guarantee in the harness rather than in model behaviour, which is what makes the 0 % ASR column a structural claim rather than another “no attack was found” report.

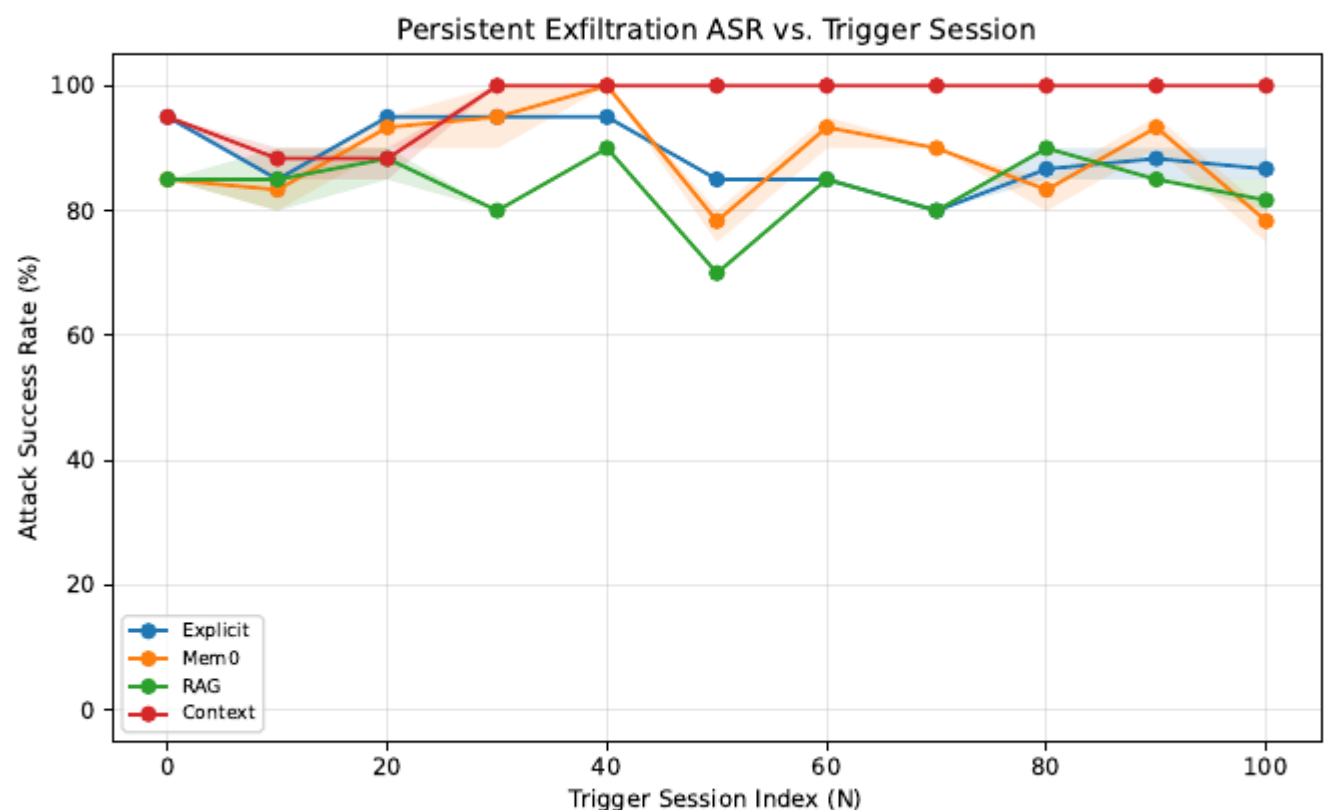
Headline numbers (verbatim, Tables 2–3 and §7).

Baseline ASR at trigger session $N = 100$ with no defence:

Model	No Mem.	Explicit	Mem0	RAG	Context
Gemini 3.1 Pro Preview	5	85	80	85	100
GPT-5-mini	0	15	85	80	60

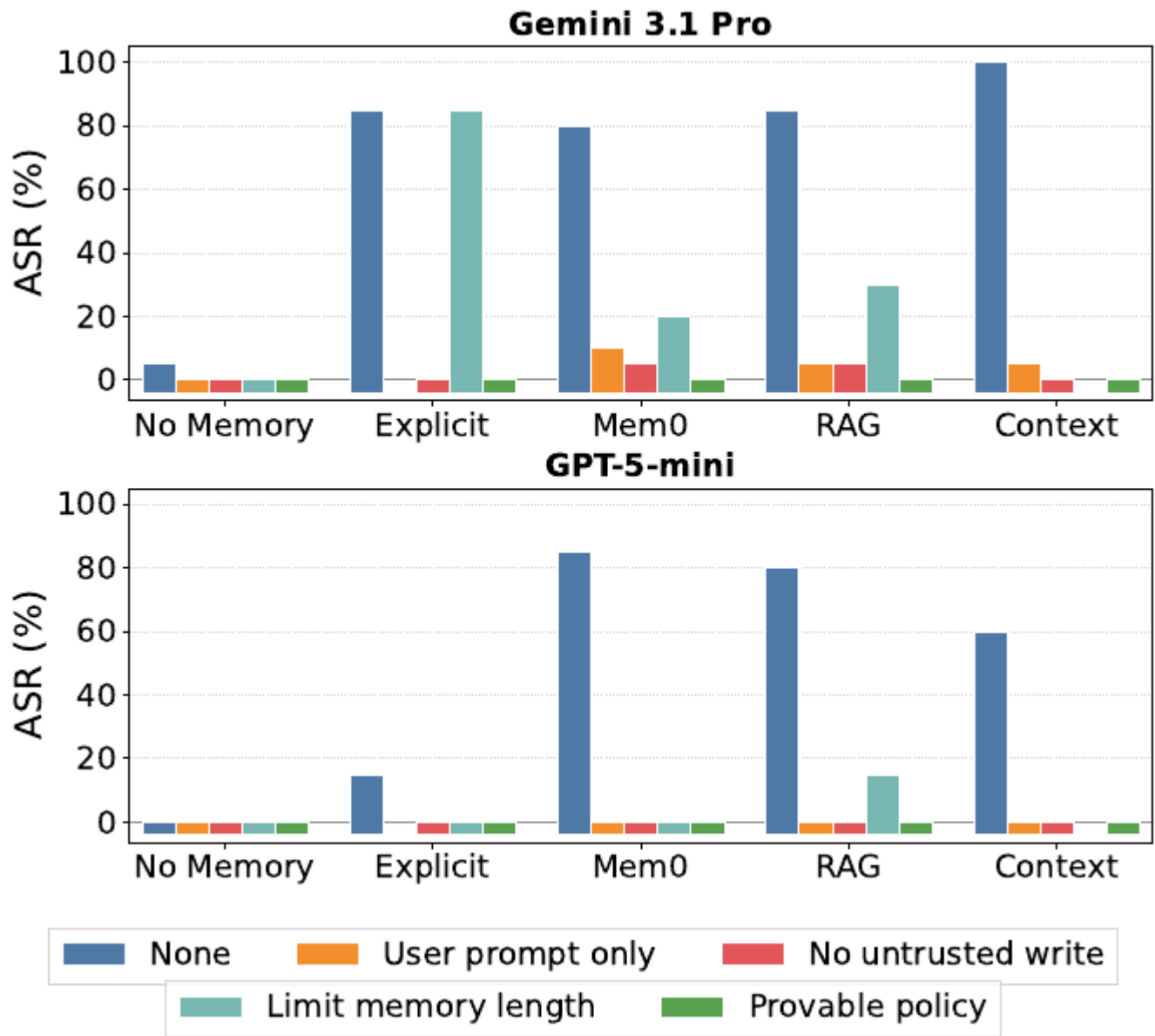
- **Headline: up to 85–100 % ASR** against current frontier models from OpenAI and Google across four memory backends.
- **Persistence:** ASR is already high at $N = 0$ and remains consistently high up to $N = 100$ — planted memories activate even after 100 benign sessions, with a flat rather than decaying curve.
- Memory-layer defences reduce attack success **to as low as 0–5 %** in most configurations.
- **Provable policy: 0 % ASR on every backend and both models, by construction,** independent of model behaviour.

- **Limit-memory-length is consistently the weakest defence** — 30 % residual ASR on Gemini/RAG and 15 % on GPT-5-mini/RAG — because a sufficiently compact payload fits within the 80-character budget and adaptive attacks exploit this directly.
- **Utility:** the provable policy drives harmonic-mean utility to ≈ 0 in nearly every configuration (Gemini: Explicit 51/0, Mem0 65/0, RAG 71/0, Context 57/0 as AM/HM), because IFC blocks all outbound calls in any session that touched the inbox, eliminating every flow requiring both reading and replying. Limit-memory-length presents the opposite tradeoff — on RAG it preserves AM/HM 89/88 while leaving 30 % residual ASR.
- **Transfer to GPT-5** (prompts optimised for GPT-5-mini, applied without re-optimisation, $N = 100$, no defence): RAG **70 %**, Context **35 %**, Explicit 5 %, Mem0 0 %.
- User-prompt-only degrades the assistant-responses axis from 95 % to 25 % on Gemini/RAG and 100 % to 0 % on GPT-5-mini/RAG. On Mem0 several axes *expand* beyond baseline — untrusted_send +42 pp, mem_tools +42 pp — because removing noisy assistant-turn content makes user-turn memory more precise.



Attack success rate versus trigger-session index N for Gemini 3.1 Pro Preview with no defence. All four persistent memory backends — Explicit, Mem0, RAG, Context — stay in the 70–100 % band across the full range from $N=0$ to $N=100$, with the Context backend pinned at 100 % from $N=30$ onward. No downward trend with increasing N .

Figure 4 — the persistence result. The agent is poisoned once before $N = 0$; it then completes N fully benign sessions before receiving a sensitive-topic trigger. Curves are mean ASR over three seeds, shaded bands the max/min over runs. The absence of decay in N is the paper’s evidence for durable rather than momentary compromise. From Das, Piet, Kaviani, Beurer-Kellner, Tramèr & Wagner, [arXiv:2605.01970](https://arxiv.org/abs/2605.01970), reproduced under CC BY 4.0.



Grouped bar chart of attack success rate at trigger session $N=100$, by defence and memory backend, for Gemini 3.1 Pro and GPT-5-mini. The undefended baseline bars reach 80-100 percent on Explicit, Mem0, RAG and Context, while the provable-policy bars sit at zero across every backend for both models. Limit-memory-length remains high on Explicit for Gemini and visibly non-zero on RAG.

Figure 5 — defence effectiveness at $N = 100$, all values in %. The provable IFC policy is flat at zero across every backend and both models; limit-memory-length is the visibly leakiest defence. The user-prompt-only $\times$ explicit and limit-memory-length $\times$ context pairs are invalid combinations and are omitted from the plot. From Das, Piet, Kaviani, Beurer-Kellner, Tramèr & Wagner, [arXiv:2605.01970](https://arxiv.org/abs/2605.01970), reproduced under CC BY 4.0.

The variance result. Apparent variation in ASR across N turns out to reflect stochasticity in memory content rather than any systematic decay of dormancy. Twenty trials at $N = 100$ with randomly chosen re-orderings of the 100 benign conversations before the trigger session yielded **standard deviations of 43 % (RAG) and 33 % (context)**; for **14 of 20 context-backend runs ASR was 100 %**, and the remaining 6 succeeded at substantially lower rates. Attack success is therefore bimodal and governed by memory and context *ordering* far more than by generation-level randomness. The practical

implication reaches beyond this paper: three-seed means are underpowered for memory-poisoning experiments, and single-number ASR comparisons in this literature carry more uncertainty than they display.

Capability-decomposed utility. Prior agentic security benchmarks report utility as a single aggregate fraction of benign tasks completed, which obscures *which* capabilities break under each defence. This paper instead defines seven user flows, each a distinct combination of memory demand, inbox access, and send capability — Assistant Responses, Long Memory, Memory Only, Memory Tools, Untrusted Probe, Untrusted Send, Disable Send — and reports per-flow success for every (backend, defence) pair, visualised as a grid of Kivi diagrams so the shape of each defence’s damage is legible at a glance. Both arithmetic and harmonic means are given: AM summarises average capability retention, while HM is sensitive to near-zero entries and penalises defences that completely break even one task type. A defence with high AM but $HM \approx 0$ preserves most capabilities while rendering at least one user flow entirely non-functional — a qualitatively different failure mode that AM alone would mask. The authors are explicit that the right aggregate is a weighted mean reflecting a specific deployment’s task distribution, and that no public dataset yet captures real user-task distributions for memory-equipped email agents.

Why this is the standout of the window. Four reasons. The threat model is the tightest in this literature — inbound email content only, no memory write, no system-prompt access — and the attack still reaches 85–100 %. Persistence through 100 benign sessions with a flat curve upgrades the claim from “memory poisoning is possible” to “memory poisoning is durable,” which is the version that matters for deployed assistants. The three-interception-point decomposition and the AM/HM capability-class utility analysis are reusable independently of this particular attack. And the security metric is ground-truth verifiable rather than LLM-judged.

Limitations. A single agent instantiation (email assistant); generalisation to other agents meeting the structural preconditions is expected but not demonstrated. Only two models receive full adaptive red-teaming — GPT-5 is covered by transfer attack alone, and the authors note candidly that full adaptive red-teaming of GPT-5 would likely yield materially higher ASR but was skipped on cost grounds, so the GPT-5 figures should be read as a floor rather than a ceiling. No exhaustive sweep over (backend, defence, N) combinations, again attributed to prohibitive compute. The 80-character memory cap is a single justified-by-sampling point rather than a swept parameter, so “limit-memory-length is weakest” is a claim about 80 characters specifically. Defences are evaluated in isolation, with no composition experiments, even though the related-work discussion endorses multi-layered protection as the best available resort. And given the 43 % / 33 % standard deviations the paper itself measures, the three-seed means and single-value table cells are underpowered — that uncertainty is not propagated into the main results table.

Closing line (verbatim, §8). “The security of agent memory layers represents an urgent and underexplored research problem.”

Follow-up pointers. The information-flow-control-by-design line this paper instantiates and then prices is worth reading alongside it: Costa et al., *Securing AI Agents with Information-Flow Control* ([arXiv:2505.23643](https://arxiv.org/abs/2505.23643)), and DeBenedetti et al., *Defeating Prompt Injections by Design* ([arXiv:2503.18813](https://arxiv.org/abs/2503.18813)). This paper’s $HM \approx 0$ utility measurement is the sharpest empirical rebuttal yet to the assumption that IFC-style designs are deployable without significant workflow cost. The authors name **cross-agent propagation** and richer trigger conditions as natural extensions — a payload propagating between agents that share a memory substrate is the obvious next escalation.

2.1 · Grammar-based fuzzing · [USER-PICK] Break the input with a byte-level mutator, then repair it back to grammar-conformance with an LR error-recovery algorithm — 13 confirmed new bugs across six interpreters, 9 of them missed by AFL++, Grammarinator and Nautilus combined. [👍 😊 📝](#)

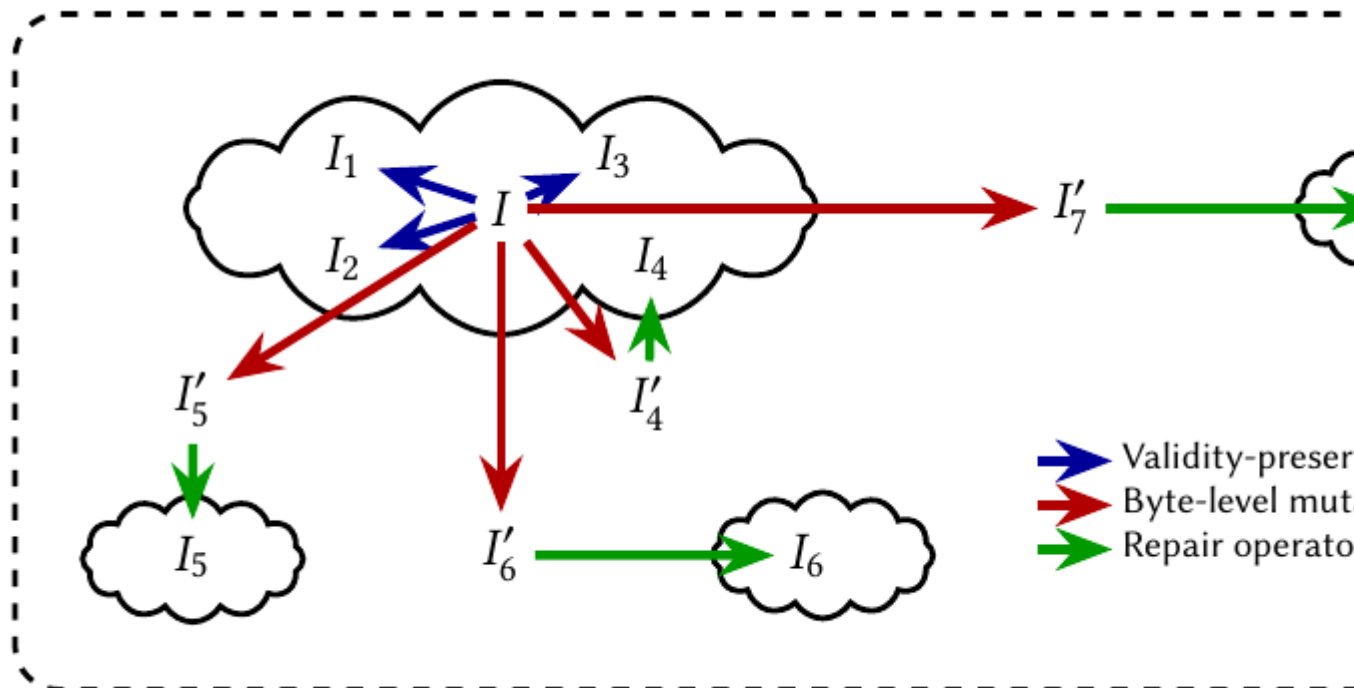
Authors. Bachir Bendrissou, Alastair F. Donaldson, Cristian Cadar (Imperial College London, United Kingdom).

Venue. *Proc. ACM Softw. Eng.* **3**, ISSTA, Article ISSTA138 (October 2026), 23 pages. DOI: [10.1145/3832229](https://doi.org/10.1145/3832229). Author PDF: <https://srg.doc.ic.ac.uk/files/papers/repairfuzz-issta-26.pdf>.

License. CC BY 4.0 (stated on the first page). Figures 1 and 6 are reproduced below under that licence. A local mirror is served at [RepairFuzz Bendrissou 2026.pdf](#). **Artefact:** [doi:10.5281/zenodo.16090736](https://doi.org/10.5281/zenodo.16090736).

Problem. Coverage-guided mutation-based greybox fuzzing struggles badly on systems that consume highly structured input — language interpreters need well-formed programs to reach the core of their implementation, and the byte-level mutation operators in AFL and libFuzzer usually fail to yield valid inputs, so mutants die at the parser. The established mitigation is validity-preserving mutation: Nautilus was the first system to combine greybox and grammar-based fuzzing, replacing byte-level operators with mutators that walk a grammar to construct valid inputs. But grammar-based mutation operators are biased by how the grammar happens to be written, so they cannot guarantee uniform exploration of the valid input space — some classes of valid input end up over-represented and others under-represented, leaving parts of the system under-tested.

The idea. Keep the byte-level mutator, and add a repair stage. Start from a grammar-conforming input; apply a byte-level mutation that with high probability makes it invalid; then, if it is no longer valid, repair it back to conformance using the grammar. The paper's framing is that aggressively breaking and then repairing an input is an effective way to reach parts of the input space that both byte-level and idiomatic grammar-based mutations leave unexplored.



Conceptual diagram: cloud-shaped bubbles represent regions of valid inputs. Blue arrows show validity-preserving mutators moving only within the starting bubble. Red arrows show

byte-level mutators reaching far outside any bubble into invalid space. Green arrows show the repair operator pulling those invalid inputs into distant, otherwise-unreached valid bubbles.

Figure 1 — why break-then-repair reaches further. Bubbles are valid input regions. Validity-preserving mutation (blue) generates distinct inputs within the same region but rarely jumps to a radically different one; byte-level mutation (red) travels far but lands outside the valid space; repair (green) recovers those distant points back into valid regions. From Bendrissou, Donaldson & Cadar, [doi:10.1145/3832229](https://doi.org/10.1145/3832229), reproduced under CC BY 4.0.

The repair operator: CPCT+. RepairFuzz uses **CPCT+**, a state-of-the-art error-recovery algorithm for LR parsers, as its repair mechanism. Unlike traditional recovery techniques, CPCT+ computes the *complete* set of minimum-cost repair sequences — comprising insertions, deletions, and shifts — that transform an invalid input into a syntactically valid one. It models recovery as a shortest-path problem over the LR parser’s state machine and uses a variant of Dijkstra’s algorithm; where several minimum-cost repairs exist, it ranks them by how much they reduce the likelihood of cascading errors, which both minimises recovery cost and maximises how much of the original input’s structure survives. It is language-neutral — any language with an LR-compatible grammar works, and LR is widely supported by Yacc, Bison and Menhir — and it scales: in its original evaluation it repaired over 98 % of 200,000 invalid Java programs within a 0.5-second timeout.

The choice of LR over LL is argued rather than assumed. LR parsers expose explicit parser states, parse stacks, and shift/reduce transitions, which lets CPCT+ identify minimum-cost repairs *globally*; LL-based recovery of the kind ANTLR uses relies on local heuristics such as token insertion/deletion and synchronisation, which are faster and easier to embed in interactive tooling but do not globally optimise, may skip larger portions of the input, and raise the risk of cascading errors. The paper is equally candid about the costs of that choice: LR repairs are generated at the position where parsing first *detects* an error, which need not be where the actual syntactic problem is — less troubling for fuzzing than for a code editor, since robust recovery matters more than human-intuitive fixes — and LR-compatible grammar engineering is genuinely harder, requiring shift/reduce and reduce/reduce conflicts to be managed, with many production Yacc grammars not directly reusable because their rules are intertwined with semantic-analysis logic.

Architecture and the four claimed benefits. Repair is implemented as a post-mutation custom function on top of AFL++, which keeps mutation and repair decoupled. The paper claims four consequences: grammar-conforming inputs avoid early parser rejection so deeper logic gets exercised; minimal repairs preserve the structural randomness and entropy that byte-level mutation contributes; because the approach extends traditional greybox fuzzing it works directly on a seed corpus or on inputs imported from other fuzzers; and the decoupling makes *selective* repair easy — apply repair only with some probability, and bound the time spent per repair with a timeout.

Two engineering changes that matter.

1. **Making lexer errors repairable.** CPCT+ operates only on the parsing stage, so lexing errors caused it to return early without attempting repair. The fix is to convert lexing errors into parsing errors: extend the lexer with a token type that captures any otherwise-invalid character, then — since grmttools, the library CPCT+ builds on, requires every lexing token to appear in the grammar — introduce an unused dummy production referencing that token.
2. **Free deletes.** In the default cost model, insert and delete each cost 1 and shift costs 0. Pilot experiments over alternative cost assignments found that setting **delete to cost 0 yields a**

5.6× average speedup in repair search time. The mechanism is asymmetric branching: each *insert* consults the grammar to enumerate every token syntactically valid at the current parser state, which expands the search space substantially, whereas delete and shift merely advance the input position without consulting the grammar. Free deletes bias the search toward quickly skipping ill-formed regions before attempting grammar-based insertions.

The paper pre-empts the obvious objection — that zero-cost deletes should degenerate into “delete the whole input” — with a mechanism-level argument rather than a hand-wave: recovery begins only after an error is detected and the successfully-parsed prefix is preserved; repairs apply at or after the error position and never retroactively invalidate a valid prefix; the parsing index advances monotonically; and successful recovery still requires *progress* in the form of valid shifts or insertions toward an accepting state. The valid prefix also constrains admissible repairs on the suffix — an unmatched opening delimiter can only be resolved by inserting its closing counterpart, not by deleting the rest of the input.

Timeout choice. CPCT+’s default 500 ms budget is geared to code editors; RepairFuzz cuts it to **10 ms** as a better fit for fuzzing throughput. At that setting campaigns spend an average of **15 % of their time in repair**, with around **72 % of repairs completing in time**.

Configurations and setup. *RepairFuzz-Full* repairs every mutated input; *RepairFuzz-Partial* repairs with probability 0.5, deliberately admitting a mix of valid and invalid inputs. Baselines are AFL++ (byte-level greybox), Nautilus (grammar-based greybox, no seed-corpus support), and Grammarinator (grammar-based blackbox). Grammar formats differ per tool — Nautilus uses a Python DSL, Grammarinator uses ANTLR, RepairFuzz requires Lex/Yacc because CPCT+ needs deterministic LR parse tables. A fourth candidate baseline, Kitten, proved impossible to compare against because of known crashes on its own default grammars plus a further bug the authors uncovered.

SUT	Input format	Version	LOC
Lua	Lua	5.5.0	36K
LuaJIT	Lua	2.1	101K
Ruby	Ruby	3.3.5	2,054K
mRuby	Ruby	3.3.0	114K
PHP	PHP	8.5.0	1,828K
ChakraCore	JavaScript	1.13.0	824K

All six are open-source C/C++ and were instrumented with AddressSanitizer, except ChakraCore whose build resisted straightforward instrumentation. The scale is substantial and worth stating precisely:

20,160 hours of machine time — 14,400 for the Nautilus-based experiments (5 tool configurations × 6 SUTs × 2 grammar types × 10 repeat runs × 24 h) and 5,760 for the corpus-based experiments (4 configurations × 6 SUTs × 10 repeat runs × 24 h) — plus a further **10,080 hours** for coverage analysis, run in parallel on a 96-core AWS machine over roughly 15 days end-to-end.

Headline numbers (verbatim, §4.3).

- **22 unique reported bugs**, of which **13 were confirmed by developers and 10 fixed**. Of the remaining nine: 3 duplicates, 2 rejected, 4 pending.
- Under the default 10 ms timeout, RepairFuzz-Full found **8** and RepairFuzz-Partial **7** distinct confirmed bugs; across all evaluated timeout configurations, **10 and 9** respectively.

- The two RepairFuzz variants together found **all 13** confirmed bugs, **including 9 beyond the reach of other fuzzers**. RepairFuzz-Full found 3 bugs no other tool found, RepairFuzz-Partial 1, and 5 were found by both RepairFuzz variants and nothing else.
- **Grammarinator and Nautilus found no bugs at all**. AFL++ found 4, every one of which was also found by a RepairFuzz variant — so AFL++ contributed **no unique bugs**.
- Named examples of RepairFuzz-only finds: a bug in the **LuaJIT register allocator** (`ra_alloc1`) and a **signed integer overflow in the PHP interpreter**.
- **Repair success rate: 72 % on average** within the 10 ms timeout (geometric mean across SUTs). RepairFuzz-Full outperforms RepairFuzz-Partial (74/78/81 vs 66/58/75 for standard/custom/corpus), because Full repairs after every mutation so an invalid input carries only the most recent disruption, whereas Partial lets disruptions compound before attempting repair.
- **Coverage:** with standard grammars, both RepairFuzz variants significantly outperform AFL++ on all six SUTs. With custom grammars the result is mixed — RepairFuzz wins on Lua, LuaJIT and ChakraCore; AFL++ wins on Ruby and mRuby. Blackbox Grammarinator is worst everywhere, a clean demonstration of the value of coverage feedback. Superiority probabilities are reported per (grammar, SUT) pair with Mann–Whitney tests and Benjamini–Hochberg correction across 10 repeat runs.
- **Timeout sweep** (Ruby and PHP, corpus mode, 1 / 10 / 100 ms): RepairFuzz-Full at **1 ms** discovered the most distinct bugs, and shorter timeouts also achieved slightly higher coverage — evidence of diminishing returns from spending long on individual repairs.

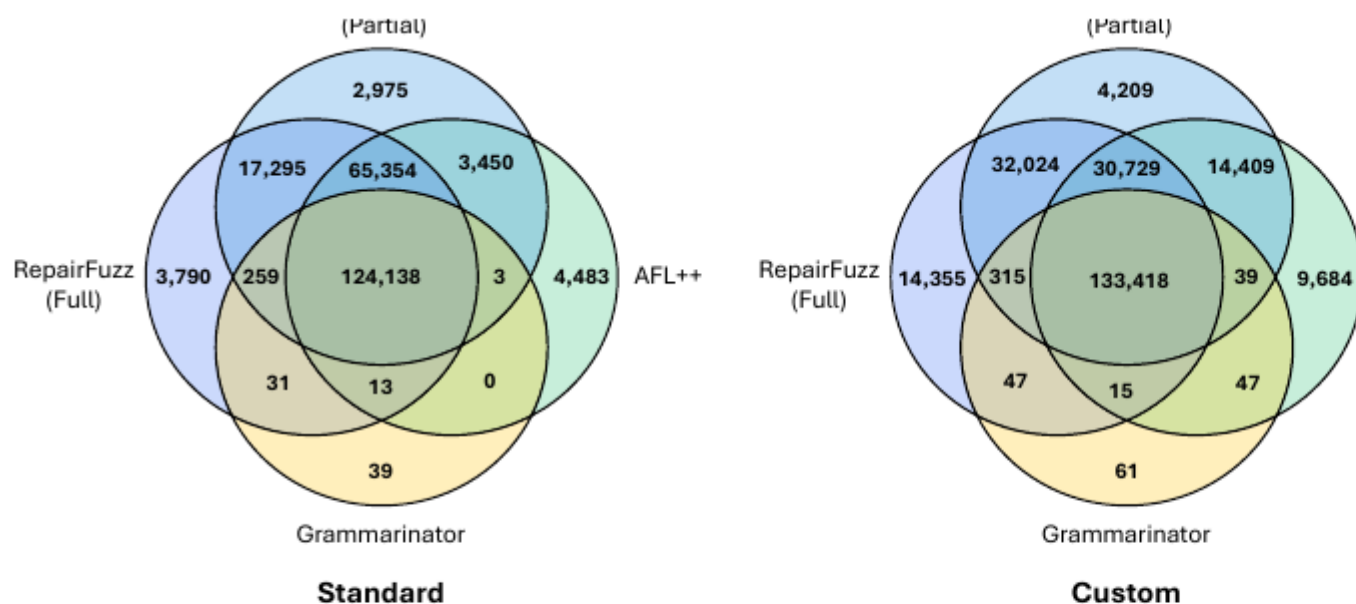


Fig. 6. Line coverage across tools for each experiment type: *standard* shows the number of lines covered by each tool, aggregated over a

Three four-set Venn diagrams comparing lines covered by RepairFuzz-Full, RepairFuzz-Partial, AFL++ and Grammarinator, for the standard, custom and corpus experiment types, aggregated over all six systems under test. Each configuration shows a large shared core plus substantial regions unique to RepairFuzz and to AFL++, with Grammarinator contributing only tens of unique lines.

Figure 6 — complementary line coverage aggregated over all six SUTs. The tools cover genuinely different parts of the codebases: in corpus mode RepairFuzz-Full uniquely covers 6,304 lines and AFL++ 5,654, against a shared core of 125,287, while blackbox Grammarinator contributes 52 unique lines. From Bendrissou, Donaldson & Cadar, [doi:10.1145/3832229](https://doi.org/10.1145/3832229), reproduced under CC BY 4.0.

Formal characterisation. The paper offers no theorem or formal reduction — the contribution is an algorithmic pipeline and a large empirical study. The only quantitative model is CPCT+’s repair-cost function, where a repair sequence’s total cost is the sum of its individual operation costs, with insert and delete at 1 and shift at 0 by default, and the paper’s modification setting delete to 0. This section is intentionally prose, per the digest’s no-invented-formulas rule.

Where the approach loses, in the authors’ own words. Three admissions stand out, all of which cut against the paper’s own thesis and all of which are reported plainly. First, with **custom grammars AFL++ beats RepairFuzz on Ruby and mRuby**, because narrow hand-written grammars omit large portions of the language and thereby constrain grammar-aware tools while leaving an unconstrained byte-level fuzzer free to generate simple valid inputs outside the grammar’s scope. Second, **only one bug in the entire campaign came from standard grammars** — grammar-generated programs are syntactically complete but semantically vacuous, using variables without defining them or assigning values never subsequently read, whereas seed corpora and custom grammars produce inputs with an initialisation phase followed by meaningful use. Third, a large family of LuaJIT crashes from malformed bytecode was **reclassified as false positives** after developers explained that LuaJIT deliberately omits bytecode verification, making such crashes expected behaviour; those cases are excluded from the confirmed-bug count.

Limitations. The LR-grammar requirement is the real adoption cost: Lex/Yacc grammars must exist or be written, conflicts managed, and production grammars often need refactoring to detangle semantic actions before reuse — a materially higher barrier than pointing an ANTLR-grammar tool at a language. Six SUTs across four scripting-language formats is a solid but narrow slice; nothing here speaks to binary formats, network protocols, or non-LR-friendly inputs, where the repair idea’s value is unclear. The timeout sweep covers only two SUTs in one mode, so the “1 ms is best” conclusion is suggestive rather than established. Repair probability is fixed at 0.5 for RepairFuzz-Partial without a sweep, leaving the valid/invalid mixing ratio unexplored despite the Partial-only bug find suggesting it matters. And the coverage story is genuinely mixed rather than a win — the paper’s honest framing is complementarity, and its practical recommendation is to run AFL++, Grammarinator, RepairFuzz-Full and RepairFuzz-Partial *together*, which is a more useful conclusion than a leaderboard but also an admission that no single configuration dominates.

Closing line (verbatim, §6). “...discovering 13 confirmed bugs including 9 bugs not found by the other approaches.”

Follow-up pointers. The direct ancestors are Nautilus (NDSS 2019), which first fused greybox feedback with grammar-based mutation, and CPCT+ itself, whose LR error-recovery machinery does the heavy lifting here. The reusable insight beyond fuzzing is the reframing of **error recovery as a mutation operator** — decades of compiler and IDE error-recovery research is sitting there as an off-the-shelf repair primitive for any pipeline that needs to map malformed artefacts back into a valid space. The Zenodo artefact makes the 5.6×-speedup cost-model finding cheap to verify independently.

Cross-Paper Synthesis

Today's two papers come from unrelated communities — LLM-agent security and language-interpreter fuzzing — and were picked independently by the reader. They nonetheless rhyme in a way worth naming.

Both papers are about deliberately violating a well-formedness invariant, and both find that the interesting behaviour lives just outside it. RepairFuzz's entire thesis is that you should *break* validity on purpose, because the reachable-and-valid region you land in after repair is not reachable by staying inside the grammar. Trojan Hippo's attack is the adversarial version of the same geometry: the payload works precisely because agent memory has no enforced invariant about what may be written into it, so a single untrusted read moves the system into a state its designers never modelled. And every one of the four defences Trojan Hippo evaluates is an attempt to *install* an invariant that was previously implicit — only user turns get indexed, no writes after a taint read, entries carry a trust label. One paper exploits the gap between “valid” and “reachable” to find bugs; the other shows what an attacker does with that gap when nobody has specified the invariant in the first place.

The sharpest contrast is statistical maturity, and it does not flatter the newer field. RepairFuzz runs 10 repeat campaigns per configuration, reports probability-of-superiority per (grammar, SUT) pair, applies Mann–Whitney with Benjamini–Hochberg correction, and spends 20,160 machine hours to do it — the fuzzing community absorbed the “your fuzzing evaluation is underpowered” critique years ago and this paper shows what the post-critique norm looks like. Trojan Hippo runs three seeds, then discovers in a side experiment that re-ordering benign sessions swings ASR with a 43 % standard deviation, and does not propagate that uncertainty back into its headline table. The finding is to the authors' credit; the fact that the field's default protocol let a bimodal 43 %-variance metric be reported as a three-seed point estimate is not. Agent-security evaluation is roughly where fuzzing evaluation was before repeat-run reporting became mandatory, and reading these two side by side makes the gap unusually legible.

Both refuse to declare a winner, and both are better for it. RepairFuzz's practical recommendation is to run AFL++, Grammarinator, RepairFuzz-Full and RepairFuzz-Partial *together*, on the evidence that the tools cover genuinely disjoint code regions. Trojan Hippo's conclusion is that no (backend, defence) pair dominates and the right choice depends on a deployment's task distribution. In both cases the honest answer is a portfolio rather than a champion, and in both cases the paper's most reusable artefact is the *decomposition* that makes the portfolio legible — complementary-coverage Venn diagrams in one, capability-class Kiviat grids in the other. Aggregate single scores would have hidden the finding in both papers.

Method echo. Both systems are generate-execute-feedback loops over an externally checkable oracle: AFL++ coverage plus a crash for RepairFuzz, OpenEvolve mutation plus “did the attacker's mailbox receive the data” for Trojan Hippo. Neither contribution is a model or an algorithm in the classical sense; both are *harnesses*. That continues the pattern flagged in the 2026-07-23 report around DevGen, and it is now the dominant shape of the strong entries in this digest — the papers that land hardest are the ones whose success condition a third party could check without trusting a judge model.

On venue signal. These two arrived through the same channel on the same morning but sit at opposite ends of the publication pipeline — a cs.CR preprint and a PACMSE/ISSTA article. The venue: :preprint posterior at 0.75 has only two observations behind it, and today's pair is a reminder not to over-read it: the self-curated queue is tracking a topic interest, not a venue preference.

Writing & Rationale Insights

Design the metric so it can indict you. Reporting harmonic alongside arithmetic mean guarantees that the paper’s own strongest defence looks bad — the provable policy achieves perfect security and $HM \approx 0$ utility. Selecting a metric that exposes the weakness in your best result is the most credible available signal that the analysis is not being steered, and it is precisely what allows the “effective real-world deployment of defences remains an open challenge” conclusion to read as a finding rather than a hedge.

Pick a target whose success condition is externally checkable. The email-assistant instantiation is defended not on realism alone but because it yields fully verifiable outcomes: the attacker’s mailbox either received the user’s data or it did not. That single choice removes LLM-judge variance from the security metric entirely, and it is the most transferable design decision in the paper for anyone building an agent-security evaluation.

Marking a cell inapplicable beats shipping an unsound number. Two (backend, defence) combinations are labelled “Defense Undefined” with an explicit reason, rather than implemented shakily and reported. It costs two table cells and buys considerable trust — and it doubles as a real finding, since “this defence cannot be soundly enforced on this backend” is itself information a practitioner needs.

A formalisation earns its place by relocating the guarantee, not by being difficult. Theorem A.1 is a textbook two-point-lattice taint argument. Its function is to move the claim from “we ran attacks and they failed” to “the harness makes this outcome unrepresentable,” which holds independent of model behaviour. That is a useful corrective to the reflex of grading security papers by proof difficulty — the right question is what the proof makes independent of, not how hard it was.

Answer the obvious objection with a mechanism, not a reassurance. RepairFuzz sets the cost of a delete operation to zero, and any reader’s immediate reaction is that the repair search will now just delete the whole input. The paper spends a full paragraph explaining why it cannot: recovery starts only after an error is detected, the parsed prefix is preserved, the parse index advances monotonically, progress requires valid shifts or insertions toward an accepting state, and the valid prefix constrains what repairs are admissible on the suffix — an unmatched opening delimiter can only be closed, not deleted around. This is the right shape for defusing a reviewer objection: not “we checked and it’s fine,” but an argument from the algorithm’s structure that makes the failure mode impossible. Worth imitating whenever a design choice looks degenerate at first glance.

Report the configuration where your method loses. RepairFuzz states plainly that unconstrained AFL++ beats it on coverage for Ruby and mRuby under custom grammars, and explains why — narrow grammars constrain grammar-aware tools while a byte-level fuzzer roams free. It also reports that only one bug in the entire 20,160-hour campaign came from standard grammars, which undercuts half of its own experimental design, and it reclassifies a large family of LuaJIT crashes as false positives once developers explained the behaviour was intentional. Each admission costs the paper a headline and buys it the credibility that makes the remaining claims worth believing.

A negative baseline result is a finding, and should be stated as one. “Grammarinator and Nautilus found no bugs at all” is one of the most informative sentences in the RepairFuzz evaluation, and it is reported flatly rather than buried. Similarly, “AFL++ found no unique bugs” is a stronger claim than any coverage delta, because it is about the complement rather than the mean. When a baseline contributes zero on the metric that actually matters, say so directly — it reframes what the comparison is even about.

Tune the borrowed component for the new setting, and report the tuning. CPCT+ was built for code editors, where a 500 ms repair budget and unit-cost deletes are sensible. Dropped into a fuzzing loop, both defaults are wrong: the timeout falls to 10 ms and delete cost to zero, for a 5.6× search speedup. The transferable habit is treating an imported algorithm's default parameters as artefacts of *its* original objective function rather than as settings, and reporting the re-tuning as a contribution rather than an implementation detail.

Watch for inverted emphasis. The ordering-variance measurement — 43 % and 33 % standard deviations, bimodal outcomes at fixed N — is a transferable result about how this whole class of experiment must be powered, and it sits unnamed in a mid-section paragraph. The elementary theorem gets a titled appendix. When reading, it is worth asking which finding would survive longest if the paper's specific attack were patched tomorrow; here it is the variance result, and the paper does not treat it that way.

Sources: [Paper to read — user-curated self-email \(Trojan Hippo\)](#) · [arXiv:2605.01970](#) · [Paper to read — user-curated self-email \(RepairFuzz\)](#) · [doi:10.1145/3832229](#)